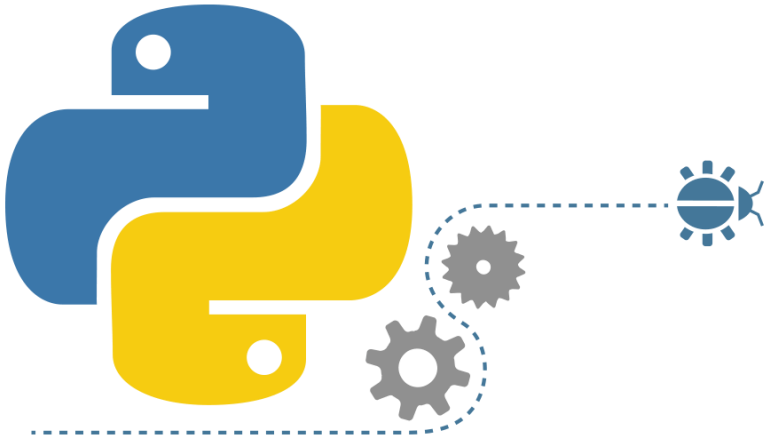
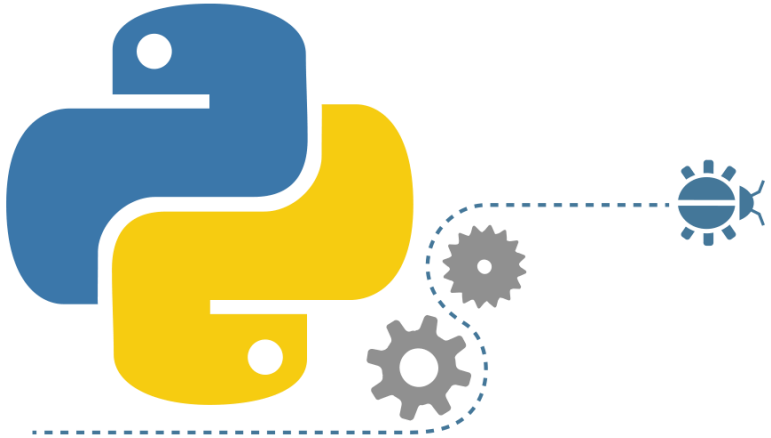
Hướng đối tượng
Object Oriented Programming

1. Lập trình hướng đối tượng trong Python
2. Tìm hiểu Class và Object trong Python
3. Kế thừa (Inheritance) trong Python
4. Đa kế thừa (Multiple Inheritance) trong Python
5. Nạp chồng toán tử trong Python
6. Import Module
7. Bài tập rèn luyện



1. Lập trình hướng đối tượng trong Python



a. Giới thiệu về OOP trong Python

- Lập trình hướng đối tượng (tiếng Anh: Object-oriented programming, viết tắt: OOP) là một kỹ thuật hỗ trợ, cho phép lập trình viên trực tiếp làm việc với các đối tượng mà họ định nghĩa lên. Hiệu quả của kĩ thuật này giúp tăng năng suất, đơn giản hoá độ phức tạp khi bảo trì cũng như mở rộng phần mềm. Hiện nay có khá nhiều ngôn ngữ lập trình theo hướng đối tượng như C++, Java, PHP,... và còn cả Python.
- Khái niệm về OOP trong Python tập trung vào việc tạo code sử dụng lại. Khái niệm này còn được gọi là DRY (Don't Repeat Yourself).

b. Các nguyên lý

- Trong Python, khái niệm về OOP tuân theo một số nguyên lý cơ bản là *tính đóng gói, tính kế thừa và tính đa hình*.
- ***Tính kế thừa (Inheritance)***: cho phép một lớp (class) có thể kế thừa các thuộc tính và phương thức từ các lớp khác đã được định nghĩa.
- ***Tính đóng gói (Encapsulation)***: là quy tắc yêu cầu trạng thái bên trong của một đối tượng được bảo vệ và tránh truy cập được từ code bên ngoài (tức là code bên ngoài không thể trực tiếp nhìn thấy và thay đổi trạng thái của đối tượng đó).
- ***Tính đa hình (Polymorphism)****:* là khái niệm mà hai hoặc nhiều lớp có những phương thức giống nhau nhưng có thể thực thi theo những cách thức khác nhau.

c. Lớp (Class) và Đối tượng (Object)

Class và Object là hai khái niệm cơ bản trong lập trình hướng đối tượng.

- **Đối tượng (Object)** là những thực thể tồn tại có hành vi.

Ví dụ đối tượng là một xe ô tô có tên hãng, màu sắc, loại nguyên liệu, hành vi đi, dừng, đỗ, nổ máy...

- **Lớp (Class)** là một kiểu dữ liệu đặc biệt do người dùng định nghĩa, tập hợp nhiều thuộc tính đặc trưng cho mọi đối tượng được tạo ra từ lớp đó.

Thuộc tính là các giá trị của lớp. Sau này khi các đối tượng được tạo ra từ lớp, thì thuộc tính của lớp lúc này sẽ trở thành các đặc điểm của đối tượng đó.

- **Phân biệt giữa Đối tượng (Object) và Lớp (Class):**
  - *Đối tượng (Object):* có trạng thái và hành vi.
  - *Lớp (Class):* có thể được định nghĩa như là một template mô tả trạng thái và hành vi mà loại đối tượng của lớp hỗ trợ. Một đối tượng là một thực thể (instance) của một lớp.

c. Lớp (Class) và Đối tượng (Object)

```
class Car:
     # thuộc tính lớp
     loaixe = "Ô tô"

     # thuộc tính đối tượng
     def __init__(self, tenxe, mausac, nguyenlieu):
        self.tenxe = tenxe
        self.mausac = mausac
        self.nguyenlieu = nguyenlieu
```

```
Porsche là Ô tô.
Toyota là Ô tô.
Lamborghini cũng là Ô tô.
Xe Toyota có màu Đỏ. Điện là nguyên liệu vận hành.
Xe Lamborghini có màu Vàng. Deisel là nguyên liệu vận hành.
Xe Porsche có màu Xanh. Gas là nguyên liệu vận hành.
```

```
# instantiate the Car class
toyota = Car("Toyota", "Đỏ", "Điện")
lamborghini = Car("Lamborghini", "Vàng", "Deisel")
porsche = Car("Porsche", "Xanh", "Gas")

# access the class attributes
print("Porsche là {}.".format(porsche.__class__.loaixe))
print("Toyota là {}.".format(toyota.__class__.loaixe))
print("Lamborghini cũng là {}.".format(lamborghini.__class__.loaixe))

# access the instance attributes
print("Xe {} có màu {}. {} là nguyên liệu vận hành.".format( toyota.tenxe, toyota.mausac, toyota.nguyenlieu))
print("Xe {} có màu {}. {} là nguyên liệu vận hành.".format( lamborghini.tenxe, lamborghini.mausac,lamborghini.nguyenlieu))
print("Xe {} có màu {}. {} là nguyên liệu vận hành.".format( porsche.tenxe, porsche.mausac, porsche.nguyenlieu))
```

d. Phương thức

Phương thức (Method) là các hàm được định nghĩa bên trong phần thân của một lớp. Chúng được sử dụng để xác định các hành vi của một đối tượng.

```
class Car:

    # thuộc tính đối tượng
    def __init__(self, tenxe, mausac, nguyenlieu):
      self.tenxe = tenxe
      self.mausac = mausac
      self.nguyenlieu = nguyenlieu

    # phương thức
    def dungxe(self, mucdich):
      return "{} đang dừng xe để {}".format(self.tenxe,mucdich)

    def chayxe(self):
      return "{} đang chạy trên đường".format(self.tenxe)

    def nomay(self):
      return "{} đang nổ máy".format(self.tenxe)
```

```
# instantiate the Car class
toyota = Car("Toyota", "Đỏ", "Điện")
lamborghini = Car("Lamborghini", "Vàng", "Deisel")
porsche = Car("Porsche", "Xanh", "Gas")

# call our instance methods
print(toyota.dungxe("nạp điện"))
print(lamborghini.chayxe())
print(porsche.nomay())
```

có ba phương thức là *dungxe(), chayxe(), nomay()*

```
Toyota đang dừng xe để nạp điện
Lamborghini đang chạy trên đường
Porsche đang nổ máy
```

e. Kế thừa (Inheritance)

Tính kế thừa cho phép một lớp (class) có thể kế thừa các thuộc tính và phương thức từ các lớp khác đã được định nghĩa. Lớp đã có gọi là lớp cha, lớp mới phát sinh gọi là lớp con. Lớp con kế thừa tất cả thành phần của lớp cha, có thể mở rộng các thành phần kế thừa và bổ sung thêm các thành phần mới.

e. Kế thừa (Inheritance)

```
# Lớp cha
class Car:

    # Constructor
    def __init__(self, hangxe, tenxe, mausac):
      # Lớp Car có 3 thuộc tính: tenxe, mausac, hang xe
      self.hangxe = hangxe
      self.tenxe = tenxe
      self.mausac = mausac

    # phương thức
    def chayxe(self):
      print ("{} đang chạy trên đường".format(self.tenxe))

    def dungxe(self, mucdich):
      print ("{} đang dừng xe để {}".format(self.tenxe, mucdich))
```

```
toyota1 = Toyota("Toyota", "Toyota Hilux", "Đỏ", "Điện")
toyota2 = Toyota("Toyota", "Toyota Yaris", "Vàng", "Deisel")
toyota3 = Toyota("Toyota", "Toyota Vios", "Xanh", "Gas")

toyota1.dungxe("nạp điện")
toyota2.chayxe()
toyota3.nomay()
```

```
Toyota Hilux đang dừng xe để nạp điện
Toyota Hilux chạy bằng Điện
Toyota Yaris đang chạy trên đường
Toyota Vios đang nổ máy
```

```
# Lớp Toyota mở rộng từ lớp Car.
class Toyota(Car):

    def __init__(self, hangxe, tenxe, mausac, nguyenlieu):
    # Gọi tới constructor của lớp cha (Car)
      # để gán giá trị vào thuộc tính của lớp cha.
        super().__init__(hangxe, tenxe, mausac)
       self.nguyenlieu = nguyenlieu

    # Kế thừa phương thức cũ
    def chayxe(self):
        print ("{} đang chạy trên đường".format(self.tenxe))

    # Ghi đè (override) phương thức cùng tên của lớp cha.
    def dungxe(self, mucdich):
       print ("{} đang dừng xe để {}".format(self.tenxe, mucdich))
       print ("{} chạy bằng {}".format(self.tenxe, self.nguyenlieu))

   # Bổ sung thêm thành phần mới
    def nomay(self):
       print ("{} đang nổ máy".format(self.tenxe))
```

f. Đóng gói (Encapsulation)

Sử dụng OOP trong Python, có thể hạn chế quyền truy cập vào trạng thái bên trong của đối tượng → ngăn chặn dữ liệu bị sửa đổi trực tiếp, được gọi là **đóng gói**

Biểu thị thuộc tính private này bằng cách sử dụng dấu gạch dưới làm tiền tố: "_" hoặc "__".

```
class Computer:

    def __init__(self):
        # Thuộc tính private ngăn chặn sửa đổi trực tiếp
        self.__maxprice = 900

    def sell(self):
        print("Giá bán sản phẩm: {}".format(self.__maxprice))

    def setMaxPrice(self, price):
        self.__maxprice = price

c = Computer()
c.sell()

# thay đổi giá.
c.__maxprice = 1000
c.sell()

# sử dụng hàm setter để thay đổi giá.
c.setMaxPrice(1000)
c.sell()
```

```
Selling Price: 900
Selling Price: 900
Selling Price: 1000
```

g. Đa hình (Polymorphism)

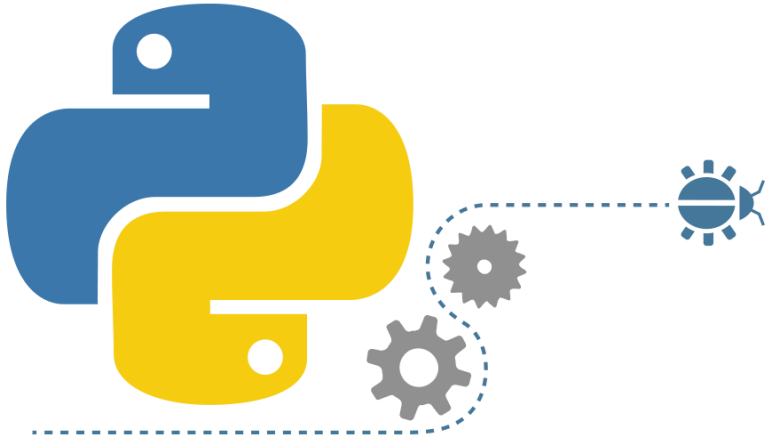
Tính đa hình là khái niệm mà hai hoặc nhiều lớp có những phương thức giống nhau nhưng có thể thực thi theo những cách thức khác nhau.

Giả sử, chúng ta cần tô màu một **hình khối**, có rất nhiều lựa chọn cho hình của bạn như **hình chữ nhật**, **hình vuông**, **hình tròn**. Tuy nhiên, bạn có thể sử dụng cùng một phương pháp để tô màu bất kỳ hình dạng nào.

```
class Toyota:

    def dungxe(self):
        print("Toyota dừng xe để nạp điện")

    def nomay(self):
        print("Toyota nổ máy bằng hộp số tự động")

class Porsche:

    def dungxe(self):
        print("Porsche dừng xe để bơm xăng")

    def nomay(self):
        print("Porsche nổ máy bằng hộp số cơ")

# common interface
def kiemtra_dungxe(car): car.dungxe()

# instantiate objects
toyota = Toyota()
porsche = Porsche()

# passing the object
kiemtra_dungxe(toyota)
kiemtra_dungxe(porsche)
```

```
Toyota dừng xe để nạp điện
Porsche dừng xe để bơm xăng
```

2. Tìm hiểu Class và Object trong Python



Nội dung bài học

Python là một ngôn ngữ lập trình hướng đối tượng. Không giống như lập trình hướng thủ tục nhấn mạnh vào các hàm, lập trình hướng đối tượng tập trung làm việc trên các đối tượng.

Đối tượng (Object) chỉ đơn giản là một tập hợp các dữ liệu (các biến) và các phương thức (các hàm) hoạt động trên các dữ liệu đó. Và, **lớp (class)** là một kế hoạch chi tiết cho đối tượng.

Chúng ta có thể nghĩ về class như một bản phác thảo (nguyên mẫu) của một ngôi nhà. Nó chứa tất cả các chi tiết về sàn nhà, cửa ra vào, cửa sổ,... Dựa trên những mô tả này, chúng ta sẽ xây dựng những ngôi nhà. Vậy nhà ở đây chính là đối tượng.

Vì nhiều ngôi nhà có thể được làm từ một mô tả nên chúng ta có thể tạo ra nhiều vật thể từ một lớp. Một đối tượng cũng được gọi là một thể hiện (**instance**) của một lớp và quá trình tạo đối tượng này được gọi là **instantiation**

a. Khai báo Class

Bắt đầu bằng một từ khóa **class**

```
class MyNewClass:
    '''Đây là docstring. Một lớp mới vừa được khai báo.'''
    pass
```

Class tạo ra một local namespace mới trở thành nơi để các thuộc tính của nó được khai báo. Thuộc tính có thể là hàm hoặc dữ liệu

Thuộc tính đặc biệt bắt đầu với dấu gạch dưới kép (__)

```
class MyClass:
    "Đây là class thứ 2 được khởi tạo"
    a = 10
    def func(self):
      print('Xin chào')

# Output: 10
print(MyClass.a)

# Output: <function MyClass.func at 0x0000000003079BF8>
print(MyClass.func)

# Output: 'Đây là class thứ 2 được khởi tạo'
print(MyClass.__doc__)
```

Ví dụ: __doc__ sẽ trả về chuỗi docstring mô tả của lớp đó

```
10
<function MyClass.func at 0x7fe90f834400>
Đây là class thứ 2 được khởi tạo
```

b. Tạo đối tượng trong Python

Đối tượng trong class có thể được sử dụng để truy cập các thuộc tính khác nhau và tạo các instance mới của lớp đó. Thủ tục để tạo một đối tượng tương tự như cách chúng ta gọi hàm.

```
ob = MyClass()
```

→ lệnh này đã tạo ra một đối tượng mới có tên là ***ob***.

Một ví dụ kĩ hơn về tạo đối tượng bao gồm cả các thuộc tính, phương thức:

```
class MyClass:
    "Đây là class thứ 3 được khởi tạo"
    a = 10
    def func(self):
      print('Xin chào')

ob = MyClass()

# Output: <function MyClass.func at 0x000000000335B0D0>
print(MyClass.func)

# Output: <bound method MyClass.func of <__main__.MyClass object at 0x000000000332DEF0>>
print(ob.func)

#  Gọi hàm func()
# Output: Xin chào
ob.func()
```

Định nghĩa hàm trong class, có parameter là ***self,*** nhưng khi gọi hàm ***obj.func()*** không cần parameter, vẫn không gặp lỗi. Bởi vì, bất cứ khi nào, object gọi các phương thức, object sẽ tự pass qua parameter đầu tiên. Nghĩa là ***obj.func()*** tương đương với ***MyClass.func(obj)***

c. Constructor trong Python

Hàm trong Class được bắt đầu với dấu gạch dưới kép **(__)** là các hàm đặc biệt, mang các ý nghĩa đặc biệt.

Một trong đó là hàm ***__init__()*** hàm này được gọi bất cứ khi nào khởi tạo một đối tượng, một biến mới trong class và được gọi là constructor trong lập trình hướng đối tượng.

```
class SoPhuc:

    def __init__(self,r = 0,i = 0):
        self.phanthuc = r
        self.phanao = i

    def getData(self):
        print("{}+{}j".format(self.phanthuc,self.phanao))

# Tạo đối tượng số phức mới
c1 = SoPhuc(2,3)

# Gọi hàm getData()
# Output: 2+3j
c1.getData()

# Tạo đối tượng số phức mới
# tạo thêm một thuộc tính mới (new)
c2 = SoPhuc(5)
c2.new = 10

# Output: (5, 0, 10)
print((c2.phanthuc, c2.phanao, c2.new))

# Đối tượng c1 không có thuộc tính 'new'
# AttributeError: 'SoPhuc' object has no attribute 'new'
c1.new
```

d. Xóa bỏ thuộc tính và đối tượng

Thuộc tính của đối tượng có thể bị xóa bằng lệnh *del.*

```
>>> c1 = SoPhuc(2,3)
>>> del c1.phanao
>>> c1.getData()
Traceback (most recent call last):
...
AttributeError: 'SoPhuc' object has no attribute 'phanao'

>>> del SoPhuc.getData
>>> c1.getData()
Traceback (most recent call last):
...
AttributeError: 'SoPhuc' object has no attribute 'getData'
```

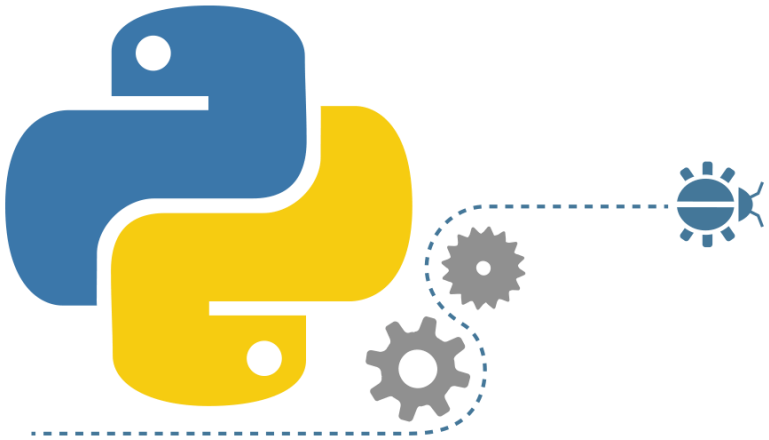
Xóa chính đối tượng đó bằng cách sử dụng câu lệnh *del*

```
>>> c1 = SoPhuc(1,3)
>>> del c1
>>> c1
Traceback (most recent call last):
...
NameError: name 'c1' is not defined
```

Sau khi bị xóa, object vẫn tồn tại trên bộ nhớ, nhưng sau đó phương thức destruction của Python (hay còn gọi là garbage collection) sẽ loại bỏ hoàn toàn các dữ liệu này trên bộ nhớ

3. Kế thừa (Inheritance) trong Python

Kế thừa trong lập trình hướng đối tượng cho phép chúng ta khai báo lớp mới sử dụng lại các hàm và thuộc tính của lớp cha cùng các chức năng mở rộng thêm.



a. Kế thừa là gì?

Kế thừa (Inheritance) cho phép một lớp (class) - kế thừa các thuộc tính và phương thức từ các lớp khác đã được định nghĩa.

Lớp đã có gọi là lớp cha (base class hoặc parent class), lớp mới phát sinh gọi là lớp con (child class hoặc derived class).

Lớp con kế thừa tất cả thành phần của lớp cha, có thể mở rộng các thành phần kế thừa và bổ sung thêm các thành phần mới.

Cú pháp của kế thừa

```
class BaseClass:
    Body of base class

class DerivedClass(BaseClass):
    Body of derived class
```

a. Kế thừa là gì?

Ví dụ: Đa giác là một hình khép kín có 3 cạnh trở lên. Chúng ta có một lớp gọi là *DaGiac* được định nghĩa như sau.

```
class DaGiac:

    def __init__(self, socanh):
        self.n = socanh
        self.canh = [0 for i in range(socanh)]

    def nhapcanh(self):
        self.canh = [float(input("Bạn hãy nhập giá trị cạnh "+str(i+1)+" : ")) for i in range(self.n)]

    def hienthicanh(self):
        for i in range(self.n):
        print("Giá trị cạnh",i+1,"là",self.canh[i])
```

```
class TamGiac(DaGiac):

    def __init__(self):
        DaGiac.__init__(self,3)

    def dientich(self):
        a, b, c = self.canh
        # Tính nửa chu vi
        s = (a + b + c) / 2
        area = (s*(s-a)*(s-b)*(s-c)) ** 0.5
        print('Diện tích của hình tam giác là %0.2f' %area)
```

Lớp ***TamGiac*** không chỉ kế thừa mà còn định nghĩa thêm một hàm mới là hàm ***dientich***.

các hàm ***nhapcanh(), hienthicanh()*** đều không có trong class TamGiac, nhưng chúng ta vẫn sử dụng được chúng

```
>>> t = TamGiac()
>>> t.nhapcanh()
Bạn hãy nhập giá trị cạnh 1 : 3
Bạn hãy nhập giá trị cạnh 2 : 5
Bạn hãy nhập giá trị cạnh 3 : 4

>>> t.hienthicanh()
Giá trị cạnh 1 là 3.0
Giá trị cạnh 2 là 5.0
Giá trị cạnh 3 là 4.0

>>> t.dientich()
Diện tích của hình tam giác là 6.00
```

b. Overriding (Ghi đè)

Python cho phép ghi đè lên các phương thức của lớp cha. Ghi đè phương thức của lớp cha nếu muốn có tính năng khác biệt hoặc đặc biệt trong lớp con.

```
def __init__(self):
    DaGiac.__init__(self,3)
```

```
def __init__(self):
    super () .__ init __ (3)
```

Kiểm tra mối quan hệ của hai lớp.

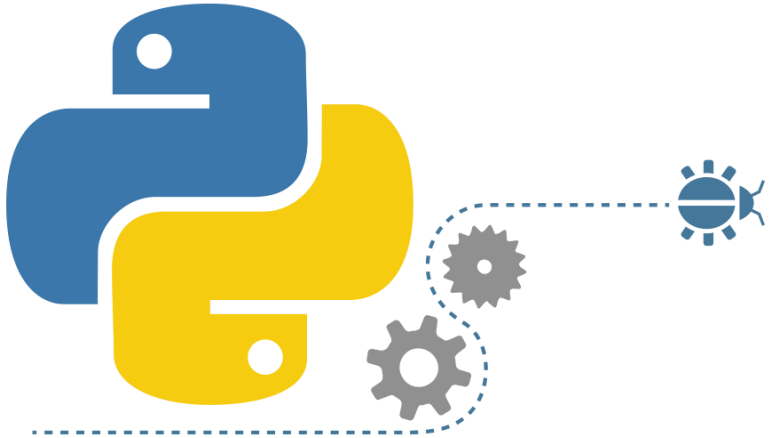
Hàm ***issubclass****(classA, classB)*
→trả về True nếu class A là lớp con của class B.

```
>>> issubclass(DaGiac,TamGiac)
False

>>> issubclass(TamGiac,DaGiac)
True

>>> issubclass(bool,int)
True
```

Hàm ***isinstance****(obj, Class)*
→ trả về True nếu obj là một instance của lớp Class hoặc là một instance của lớp con của Class.

```
>>> isinstance(t,TamGiac)
True

>>> isinstance(t,DaGiac)
True

>>> isinstance(t,int)
False

>>> isinstance(t,object)
True
```

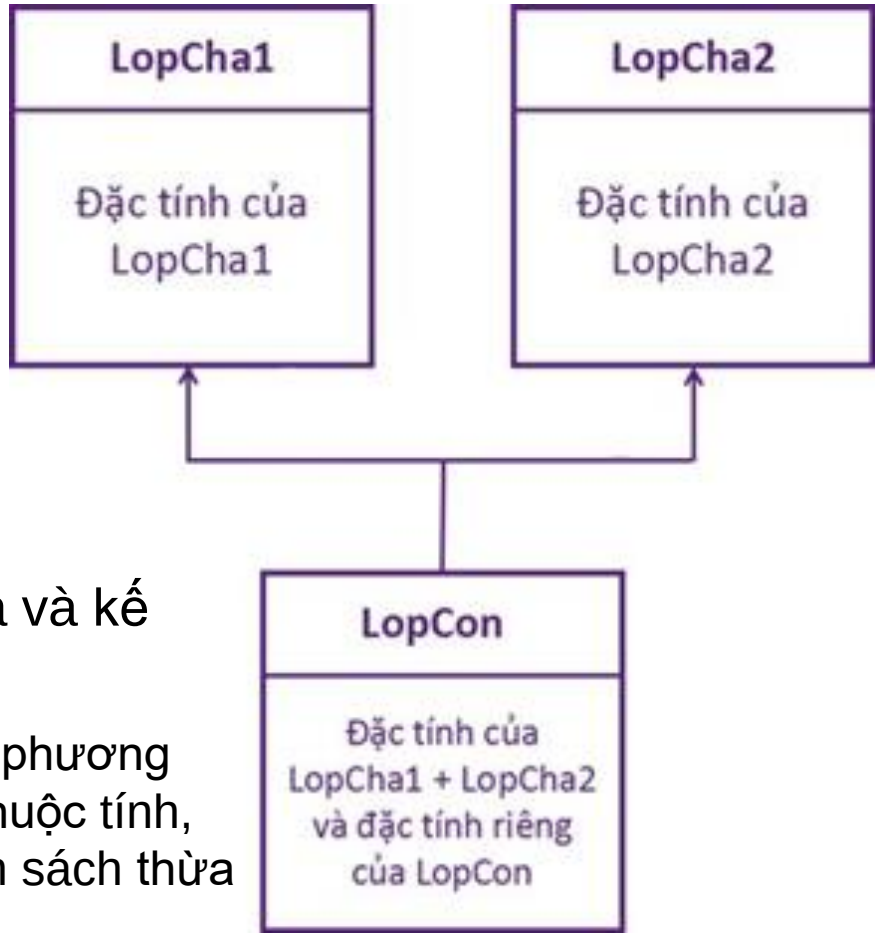
4. Đa kế thừa (Multiple Inheritance) trong Python



a. Đa kế thừa (Multiple Inheritance)

Cú pháp:

```
class LopCha1:
    pass

class LopCha2:
    pass

class LopCon(LopCha1, LopCha2):
    pass
```



Lớp con được định nghĩa từ nhiều lớp cha và kế thừa đặc tính của cả hai lớp.

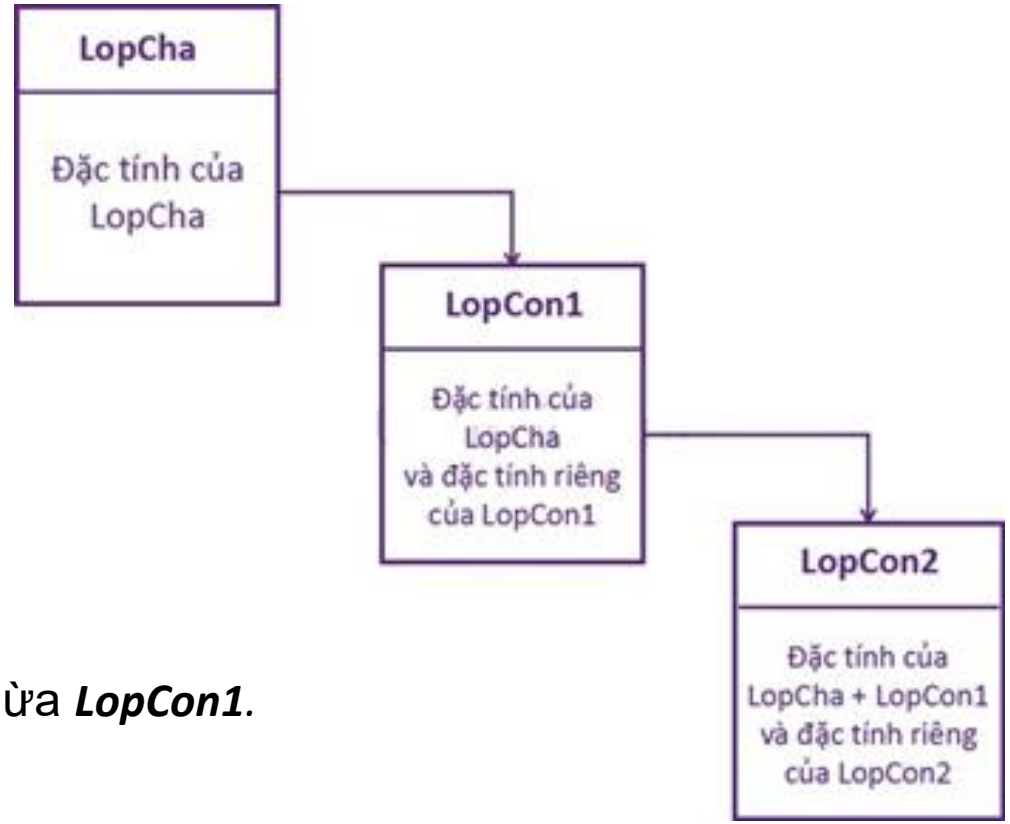
Các lớp cha có thể có các thuộc tính hoặc các phương thức giống nhau. Lớp con sẽ ưu tiên thừa kế thuộc tính, phương thức của lớp đứng đầu tiên trong danh sách thừa kế

b. Kế thừa đa cấp (Multilevel Inheritance)

Ngoài việc có thể kế thừa từ các lớp cha, bạn còn có thể tạo lớp con mới kế thừa các lớp con trước đó. Đây gọi là kế thừa đa cấp (Multilevel Inheritance).

Cú pháp:

```
class LopCha:
    pass

class LopCon1(LopCha):
    pass

class LopCon2(LopCon1):
    pass
```



LopCon1 kế thừa ***LopCha***, ***LopCon2*** kế thừa ***LopCon1***.

c. Thứ tự truy xuất phương thức (Method Resolution Order - MRO)

Class được bắt nguồn từ object. Trong kịch bản đa thừa kế, bất kỳ thuộc tính cần được truy xuất nào, đầu tiên sẽ được tìm kiếm trong lớp hiện tại. Nếu không tìm thấy, tìm kiếm tiếp tục vào lớp cha đầu tiên và từ trái qua phải.

→ Thứ tự truy xuất sẽ là *[LopCon, LopCha1, LopCha2, object].*

Thứ tự này còn được gọi là tuyến tính hóa của *LopCon* và tập hợp các quy tắc được sử dụng để tìm thứ tự này được gọi là Thứ tự truy xuất phương thức (MRO).

Nói một cách dễ hiểu, MRO dùng để hiển thị danh sách/tuple các class cha của một class nào đó.

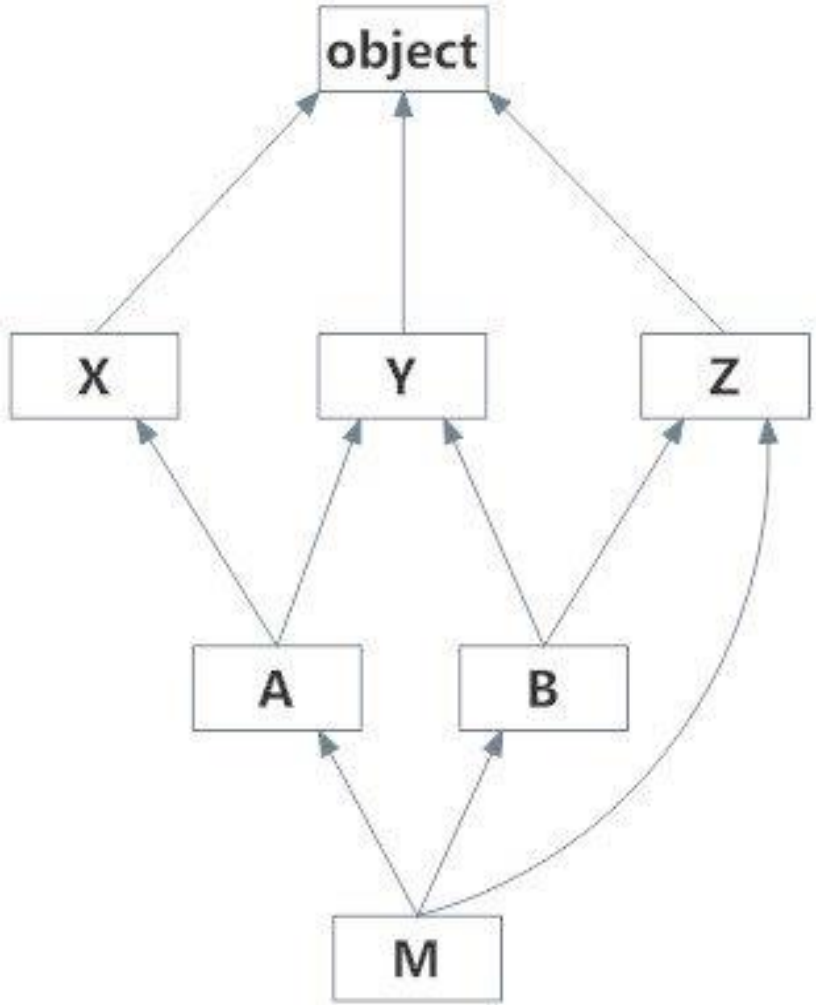
MRO được sử dụng theo hai cách:

- __mro__: trả về một tuple
- mro(): trả về một danh sách

```
>>> LopCon.__mro__
(<class '__main__.LopCon'>,
 <class '__main__.LopCha1'>,
 <class '__main__.LopCha2'>,
 <class 'object'>)

>>> LopCon.mro()
[<class '__main__.LopCon>,
 <class '__main__.LopCha1'>,
 <class '__main__.LopCha2'>,
 <class 'object'>]
```

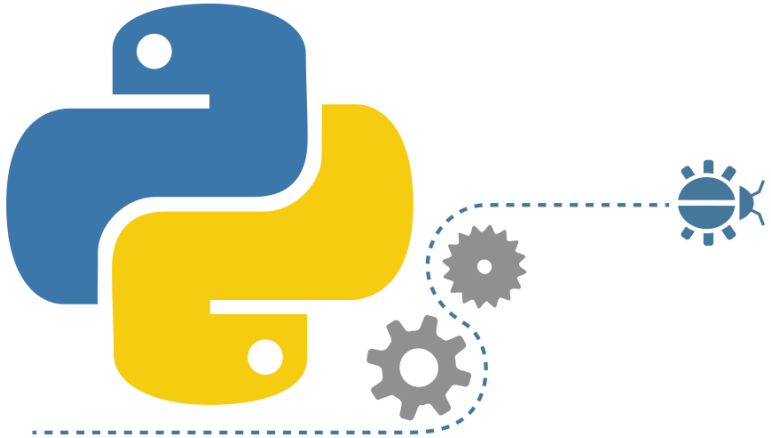
c. Thứ tự truy xuất phương thức (Method Resolution Order - MRO)

Ví dụ thừa kế phức tạp và hiển thị trực quan của nó cùng với MRO




```
class X: pass
class Y: pass
class Z: pass

class A(X,Y): pass
class B(Y,Z): pass

class M(B,A,Z): pass

# Output:
# [<class '__main__.M'>, <class '__main__.B'>,
# <class '__main__.A'>, <class '__main__.X'>,
# <class '__main__.Y'>, <class '__main__.Z'>,
# <class 'object'>]
```

5. Nạp chồng toán tử trong Python



a. Nạp chồng toán tử trong Python là gì?

Toán tử Python làm việc bằng các hàm được dựng sẵn, nhưng một toán tử có thể được sử dụng để thực hiện nhiều hoạt động khác nhau. Ví dụ với toán tử ' + ', bạn có thể cộng số học hai số với nhau, có thể kết hợp hai danh sách, hoặc nối hai chuỗi khác nhau lại…

Tính năng này trong Python gọi là nạp chồng toán tử, cho phép cùng một toán tử được sử dụng khác nhau tùy từng ngữ cảnh.

Vậy vấn đề gì sẽ xảy ra khi ta sử dụng nạp chồng toán tử với object của một lớp do người dùng khai báo? Hãy theo dõi ví dụ mô phỏng một điểm trong hệ tọa độ hai chiều sau:

```
class Point:
    def __init__(self, x = 0, y = 0):
      self.x = x
      self.y = y
```

Ta chạy chương trình và nhập vào các điểm:

```
>>> p1 = Point(2,3)
>>> p2 = Point(-1,2)
>>> p1 + p2
Traceback (most recent call last):
...
TypeError: unsupported operand type(s) for +: 'Point' and 'Point'
```

Báo lỗi *TypeError* vì Python không thể nhận hai đối tượng *Point* cùng lúc.
Để xử lý vấn đề này, ta sẽ sử dụng nạp chồng toán tử

b. Nạp chồng toán tử ' + ' trong Python

Để nạp chồng toán tử ' + ', ta sẽ sử dụng hàm *__add__()* trong class.
Ta có thể triển khai nhiều công việc bằng hàm này, ví dụ như cộng hai điểm tọa độ ở ví dụ bên trên

```
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x,self.y)

    def __add__(self,other):
        x = self.x + other.x
        y = self.y + other.y
        return Point(x,y)
```

```
>>> p1 = Point(2,3)
>>> p2 = Point(-1,2)
>>> print(p1 + p2)
(1,5)
```

b. Nạp chồng toán tử ' + ' trong Python

TOÁN TỬ	BIỂU DIỄN	HOẠT ĐỘNG
Phép cộng	p1 + p2	p1.__add__(p2)
Phép trừ	p1 - p2	p1.__sub__(p2)
Phép nhân	p1 * p2	p1.__mul__(p2)
Lũy thừa	p1 ** p2	p1.__pow__(p2)
Phép chia	p1 / p2	p1.__truediv__(p2)
Phép chia lấy phần nguyên (Floor Division)	p1 // p2	p1.__floordiv__(p2)
Số dư (modulo)	p1 % p2	p1.__mod__(p2)
Thao tác trên bit: phép dịch trái	p1 << p2	p1.__lshift__(p2)
Thao tác trên bit: phép dịch phải	p1 >> p2	p1.__rshift__(p2)
Thao tác trên bit: phép AND	p1 & p2	p1.__and__(p2)
Thao tác trên bit: phép OR	p1 | p2	p1.__or__(p2)
Thao tác trên bit: phép XOR	p1 ^ p2	p1.__xor__(p2)
Thao tác trên bit: phép NOT	~p1	p1.__invert__()

c. Nạp chồng toán tử so sánh trong Python

Có nhiều toán tử so sánh được hỗ trợ bởi Python, ví dụ như: <, >, <=, >=, ==,...

Bạn sử dụng nạp chồng toán tử này khi muốn so sánh các đối tượng trong lớp với nhau.

Ví dụ bạn muốn so sánh các điểm trong class *Point,* hãy so sánh độ lớn của các điểm này bắt đầu từ gốc tọa độ, thực hiện như sau:

```
class Point:
    def __init__(self, x = 0, y = 0):
        self.x = x
        self.y = y

    def __str__(self):
        return "({0},{1})".format(self.x,self.y)

    def __lt__(self,other):
        self_mag = (self.x ** 2) + (self.y ** 2)
        other_mag = (other.x ** 2) + (other.y ** 2)
        return self_mag < other_mag
```

```
>>> Point(1,1) < Point(-2,-3)
True

>>> Point(1,1) < Point(0.5,-0.2)
False

>>> Point(1,1) < Point(1,1)
False
```

c. Nạp chồng toán tử so sánh trong Python

TOÁN TỬ	BIỂU DIỄN	HOẠT ĐỘNG
Nhỏ hơn	p1 < p2	p1.__lt__(p2)
Nhỏ hơn hoặc bằng	p1 <= p2	p1.__le__(p2)
Bằng	p1 == p2	p1.__eq__(p2)
Khác	p1 != p2	p1.__ne__(p2)
Lớn hơn	p1 > p2	p1.__gt__(p2)
Lớn hơn hoặc bằng	p1 >= p2	p1.__ge__(p2)

d. Các hàm đặc biệt trong Python

Hàm trong Class được bắt đầu với hai dấu gạch dưới liền nhau *(__)* là các hàm đặc biệt, mang các ý nghĩa đặc biệt. Có rất nhiều hàm đặc biệt trong Python và một trong đó là hàm *__init__()*, hàm này được gọi bất cứ khi nào khởi tạo một đối tượng, một biến mới trong class.

Mục đích khi sử dụng các hàm đặc biệt này là giúp những hàm của chúng ta tương thích với các hàm được dựng sẵn trong Python.

```
>>> p1 = Point(2,3)
>>> print(p1)
<__main__.Point object at 0x00000000031F8CC0>
```

Nên khai báo phương thức *__str__()* trong class để kiểm soát cách hiển thị kết quả được in ra

```
class Point:
    def __init__(self, x = 0, y = 0):
      self.x = x
      self.y = y

    def __str__(self):
      return "({0},{1})".format(self.x,self.y)
```

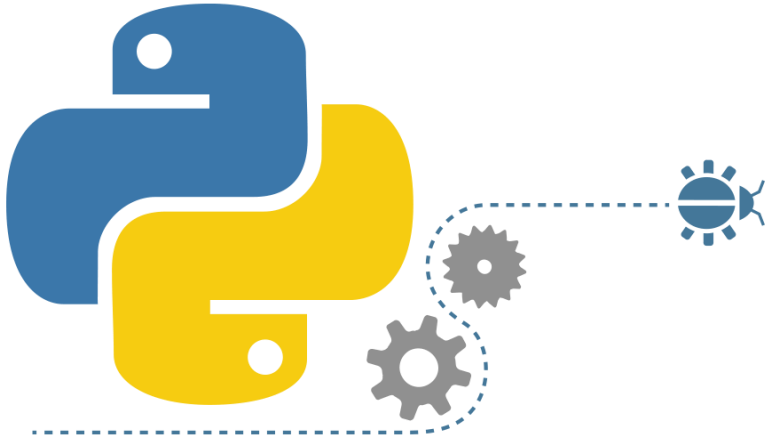
```
>>> p1 = Point(2,3)
>>> print(p1)
(2,3)
```

__str__() làm kết quả hiển thị chuẩn hơn

```
>>> str(p1)
'(2,3)'

>>> format(p1)
'(2,3)'
```

sử dụng hàm tích hợp sẵn trong Python là **str**() và **format**() lệnh thực hiện gọi p1.__str__() nên kết quả được trả về tương tự

6. Import Module



a. Module trong Python là gì?

Module (mô-đun) đề cập đến một file chứa những câu lệnh Python và các định nghĩa. Một file chứa code Python, ví dụ ***mymodule.py*** được gọi là module và tên của module sẽ là ***mymodule***.

Module thường được sử dụng khi muốn chia chương trình lớn thành những file nhỏ hơn để dễ quản lý và tổ chức. Phổ biến nhất là những hàm Python hay phải sử dụng sẽ được định nghĩa trong một module và nhập vào Python thay vì sao chép định nghĩa trong những chương trình khác nhau. Nhờ thế, module cho phép tái sử dụng code.

Giờ thử tạo một module bằng cách nhập code sau vào file và lưu với tên ***mymodule.py***.

```
#Ví dụ về module Python
def them(p, q):
    """Module này thêm 2 số
    và trả về kết quả """

    ket_qua = p + q
    return ket_qua
```

Ở đây, chúng ta định nghĩa hàm them() trong module mymodule. Hàm sẽ lấy vào 2 số và trả về tổng của chúng.

b. Làm sao để nhập module trong Python?

Sử dụng từ khóa **import**

```
>>> import mymodule
```

Lệnh này không nhập tên của hàm được định nghĩa trong `mymodule` một cách trực tiếp trong bảng hiện tại, nó chỉ nhập tên của module mà thôi. Sử dụng tên module để truy cập vào hàm với toán tử (.). Ví dụ:

```
>>> mymodule.them(4.5,5.5)
10.0
```

Hãy chắc chắn bạn đã gõ đúng tên module, tên hàm (phân biệt chữ hoa, chữ thường) nếu bạn nhận được thông báo sau: "ModuleNotFoundError: No module named..."

Có sẵn rất nhiều module tiêu chuẩn. Bạn có thể kiểm tra danh sách đầy đủ tại địa chỉ: **https://docs.python.org/3/py-modindex.html**.

b1. Sử dụng lệnh import

Chúng ta có thể sử dụng lệnh import để nhập module và truy cập vào các định nghĩa bên trong nó, sử dụng toán tử . như mô tả bên trên. Đây là ví dụ:

```
# Lệnh import để nhập module có sẵn trong Python
import math
print("Giá trị của pi là: ", math.pi)
```

```
Giá trị của pi là: 3.141592653589793
```

b2. Nhập module và sửa tên khi nhập

```
# Nhập module và sửa tên nó
import math as m
print("Giá trị của pi là: ", m.pi)
```

Đổi tên module **math** là **m**, giúp tiết kiệm thời gian trong một số trường hợp.

Chú ý, việc đổi tên này chỉ áp dụng trong phạm vi lệnh, chứ không thực sự đổi tên module trong Lib.

Khi đã đổi tên, bạn phải gõ đúng tên module, **math** lúc này không được công nhận trong phạm vi lệnh nữa, mà bạn phải dùng **m** mới đúng

b3. Lệnh from...import trong Python

Chúng ta có thể nhập một tên cụ thể từ module mà không cần nhập toàn bộ module như ví dụ dưới đây:

```
# Chỉ nhập pi từ module math
from math import pi
print("Giá trị của pi là: ",pi)
```

Code trên chỉ nhập thuộc tính pi từ module math. Trong trường hợp này không cần sử dụng toán tử "."

Có thể nhập nhiều thuộc tính của module như ví dụ dưới đây:

```
# Nhập nhiều thuộc tính từ module math
>>> from math import pi, e
>>> pi
3.141592653589793
>>> e
2.718281828459045
```

b4. Nhập tất cả tên

Ta có thể nhập tất cả tên (các định nghĩa) từ một module sử dụng code sau:

```
# Nhập tất cả tên từ module math
form math import *
print("Giá trị của pi là: ",pi)
```

Chúng ta nhập tất cả các định nghĩa từ module math nên tất cả tên đều có thể nhìn thấy trong phạm vi này, ngoại trừ những tên bắt đầu bằng dấu gạch dưới _.

Nhập mọi thứ với dấu hoa thị * không phải là một thói quen lập trình tốt. Vì nó có thể dẫn đến những định nghĩa bị trùng lặp cho cùng một định danh và khiến cho việc đọc code trở nên khó khăn hơn

c. Đường dẫn tìm kiếm module Python

Khi nhập module, Python sẽ tìm một vài nơi. Trình thông dịch tìm các module có sẵn, nếu không thấy nó sẽ vào danh sách các thư mục được định nghĩa trong sys.path. Thứ tự tìm kiếm sẽ là:

- Thư mục hiện tại.
- PYTHONPATH (một biến môi trường với danh sách thư mục).
- Thư mục mặc định có vị trí phụ thuộc vào chọn lựa trong quá trình cài đặt.

Nhập lần lượt các lệnh sau để xem đường dẫn:

```
>>> import sys
>>> sys.path
```

Chúng ta có thể chỉnh sửa danh sách này để thêm các path tùy chỉnh theo mong muốn.

d. Nạp lại module

Trình thông dịch Python chỉ nhập một module trong một phiên. Điều này làm cho mọi thứ trở nên hiệu quả hơn. Dưới đây là ví dụ minh họa cho cách hoạt động này.

Bạn viết code sau trong file mới, đặt tên là **module_1**:

```
print("Code đã hoạt động")
```

Giờ ta thử nhập module module_1 để xem sự hiệu quả của việc nhập module nhiều lần.

```
>>> import module_1
Code đã hoạt động
>>> import module_1
>>> import module_1
>>> import module_1
```

Code trên chỉ thực hiện một lần, nghĩa là `module_1` chỉ được nhập 1 lần trong 1 phiên làm việc đó. Bây giờ nếu sửa đổi code trong `module_1` thì phải nạp lại nó, phải khởi động lại trình thông dịch, nhưng cách này có vẻ không ổn lắm.
Python cung cấp cách thông minh hơn để làm điều này. Bạn có thể sử dụng hàm `reload()` trong module imp để nạp lại `module_1`, như dưới đây:

```
>>> import module_1
Code đã hoạt động
>>> import module_1
>>> import module_1
>>> import imp
>>> imp.reload(module_1)
Code đã hoạt động
<module 'module_1' from
'C:/Users/myusers/Python/Python36-32\\module_1.py'>
```